

aligned with the "less is more" philosophy as discussed in the [LIMA paper](#), a subject I previously covered in a previous issue of this newsletter:



Following the initial supervised finetuning, the models were further refined using RLHF. While an in-depth discussion of RLHF is beyond the scope of this already extensive article, further information can be found in a previous issue of this newsletter as well:



It is worth mentioning that InstructGPT and ChatGPT leveraged proximal policy optimization (PPO) for RLHF training. In contrast, the Llama 2 authors conducted experiments with two different methods, [rejection sampling](#) and RLHF with PPO.

The authors also nicely illustrated the evolution of the Llama 2 70B Chat models, tracing their journey from the initial supervised finetuning (SFT-v1) to the final RLHF finetuning stage with PPO (RLHF-v5). The chart reflects consistent improvements in both the harmlessness and helpfulness axes, as shown in the annotated plots below.

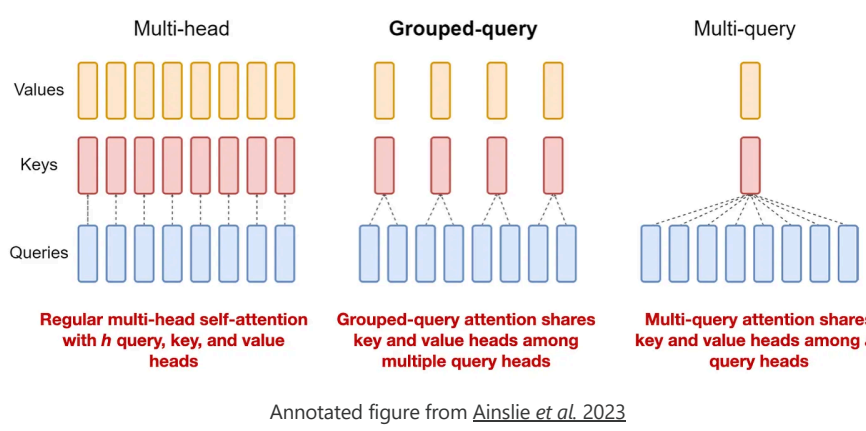


In the future, it will be interesting how [Direct Policy Optimization \(DPO\)](#), a reinforcement learning-free alternative I previously covered in [AI Research Highlights In 3 Sentences Or Less \(May-June 2023\)](#), compares to these RLHF approaches.

Llama 2 Implementation Details and Interesting Tidbits

The Llama 2 paper is very detailed, and covering all its aspects in this newsletter issue would be impossible. However, below, I wanted to highlight a few more tidbits that I found interesting.

For instance, 34B and 70B parameter Llama models use [grouped-query attention \(GQA\)](#). GQA can be regarded as a more generalized form of multi-query attention, a concept previously employed in models such as Falcon.



The likely motivation behind this is to reduce computational requirements with minimal impacts on the modeling performance.

Do we still need to finetune models?

While the Llama 2 base and chat models perform well, there's still room for improvement through finetuning. Think of Llama 2 as a revised foundational model that can be further finetuned for more specialized tasks.

To illustrate the potential benefit of finetuning, let's take the example of the BoolQ benchmark. Surprisingly, Llama 2 doesn't outperform DeBERTa-1.5B, an encoder-only model, on this particular task. It's a reminder that even strong models like Llama 2 can be outperformed by smaller finetuned models, especially when targeting specific tasks.

[illegible]

For fairness, the DeBERTa-1.5B model (initially proposed at ICLR 2021) was finetuned on the training data portion of BoolQ, whereas Llama 2 was used via few-shot prompting.

While this experiment has not been done yet, a finetuned Llama 2 model could perform even better on this dataset -- or at least better than the Llama 2 base model.

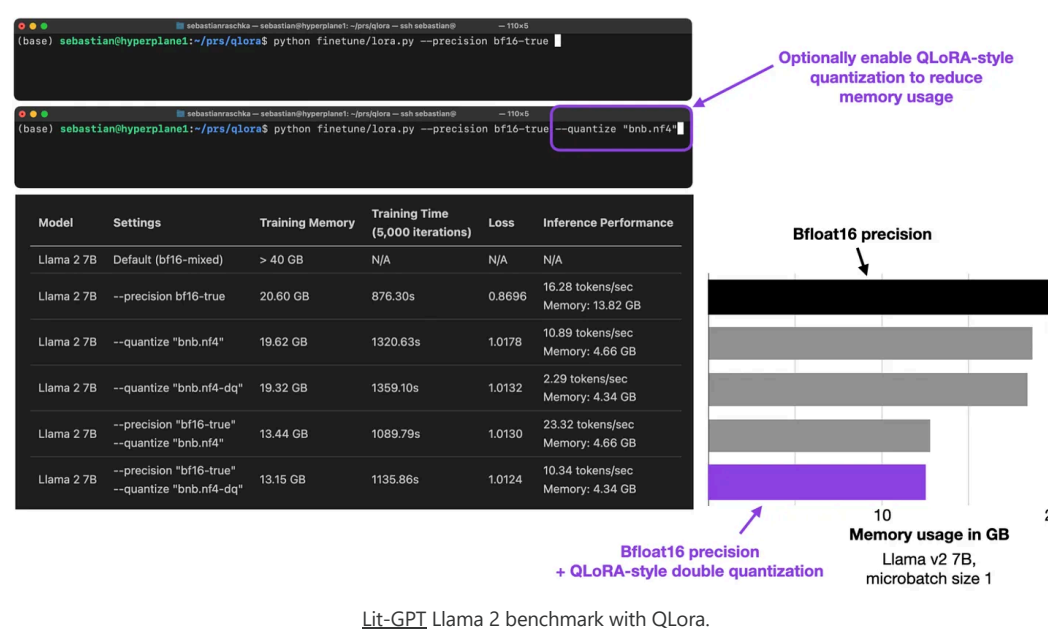
(And as shown via [DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing](#) at ICLR 2023, it's also possible to even use a much smaller 300M parameter model for good performance on this task.)

Using Llama 2

As we've seen above, Llama 2 is an extremely capable model. However, how do we use Llama 2 in a custom project?

Meta maintains [a GitHub repository](#) with the inference code to use the Llama 2 weights.

In addition, Llama is now also supported in Lit-GPT, an open-source repository that supports a large variety of LLMs for pretraining and finetuning. Among others, I recently helped implement QLoRA support in Lit-GPT and included a few Llama 2 performance benchmarks below.



Andrej Karpathy also released a GitHub repository for Llama 2 inference (not pretraining or finetuning) in just plain C code, which can be interesting for tinkers who want to just run but not finetune Llama 2.

Code Llama

Llama 2 is awesome. However, coding tasks were not its strong suit. For instance, the HumanEval benchmark is a coding-related benchmark from the paper [Evaluating Large Language Models Trained on Code](#).

The screenshot shows the OpenAI Playground interface. At the top, the 'Model' dropdown is set to 'LLaMA 2 (70B)'. Below the input area, the 'Download:' section shows 'GPT-4' as the selected model, with 'GPT-3.5' and 'Other models' as options. A table of benchmark results is displayed, comparing LLaMA 2 against other models across various tasks.

Benchmark (shots)	GPT-3.5	GPT-4	PaLM	LLaMA 2	LLaMA 2
MMLU (5-shot)	70.0	86.4	69.3	78.3	68.9
TriviaQA (1-shot)	-	81.4	86.3	-	83.0
Natural Questions (1-shot)	-	29.3	37.5	35.0	35.0
CoSQA (8-shot)	37.7	92.0	56.5	30.7	56.8
HumanEval (0-shot)	48.1	67.0	26.2	-	29.9
Big-Bench Hard (3-shot)	-	52.3	65.7	51.2	-

Annotated figure from the original [Llama 2 paper](#)

Two days ago, [Meta released a new suite of code models](#) (7B, 13B, and 34B), under the same license as Llama 2, that have been specifically trained on code. These models come in three flavors: a general code model (Code Llama), an instruction-following model (Code Llama-instruct), and a version specialized to Python code (Code Llama-Python). Note that all Code Llama models were initialized with Llama 2 weights before they were further trained on code.

The 34B Code Llama model is about twice as good as the original 70B Llama 2 model, closing the gap to (the much larger) GPT-4.

possible means that the model is allowed to generate **coarse outputs**, and a problem is considered solved if one of these passes the unit tests.

Model	Size	Headword	Size	MBPP
code-llama-001	12B	33.3%	100.0%	65.9%
code-llama-001-32k	12B	33.3%	100.0%	65.9%
code-llama-001-32k-chat	12B	67.6%	100.0%	67.6%
PALM-CoCo	540B	35.5%	88.4%	36.0%
PALM-CoCo 2.5	540B	35.5%	88.4%	36.0%
BaseCodeFlan	15.3B	30.0%	89.0%	49.0%
BaseCodeFlan-2	15.3B	30.0%	89.0%	49.0%
BaseCodeFlan-3	15.3B	30.0%	89.0%	49.0%
BaseCodeFlan-Pretrained	15.3B	48.8%	89.0%	55.5%
LEANA 2				
LEANA 2	120B	26.1%	44.0%	48.5%
LEANA 2-16	120B	26.1%	44.0%	48.5%
LEANA 2-32	120B	26.1%	44.0%	48.5%
LEANA 2-64	120B	26.1%	44.0%	48.5%
LEANA 2-128	120B	26.1%	44.0%	48.5%
LEANA 2-256	120B	26.1%	44.0%	48.5%
LEANA 2-512	120B	26.1%	44.0%	48.5%
LEANA 2-1024	120B	26.1%	44.0%	48.5%
LEANA 2-2048	120B	26.1%	44.0%	48.5%
LEANA 2-4096	120B	26.1%	44.0%	48.5%
LEANA 2-8192	120B	26.1%	44.0%	48.5%
LEANA 2-16384	120B	26.1%	44.0%	48.5%
LEANA 2-32768	120B	26.1%	44.0%	48.5%
LEANA 2-65536	120B	26.1%	44.0%	48.5%
LEANA 2-131072	120B	26.1%	44.0%	48.5%
LEANA 2-262144	120B	26.1%	44.0%	48.5%
LEANA 2-524288	120B	26.1%	44.0%	48.5%
LEANA 2-1048576	120B	26.1%	44.0%	48.5%
LEANA 2-2097152	120B	26.1%	44.0%	48.5%
LEANA 2-4194304	120B	26.1%	44.0%	48.5%
LEANA 2-8388608	120B	26.1%	44.0%	48.5%
LEANA 2-16777216	120B	26.1%	44.0%	48.5%
LEANA 2-33554432	120B	26.1%	44.0%	48.5%
LEANA 2-67108864	120B	26.1%	44.0%	48.5%
LEANA 2-134217728	120B	26.1%	44.0%	48.5%
LEANA 2-268435456	120B	26.1%	44.0%	48.5%
LEANA 2-536870912	120B	26.1%	44.0%	48.5%
LEANA 2-1073741824	120B	26.1%	44.0%	48.5%
LEANA 2-2147483648	120B	26.1%	44.0%	48.5%
LEANA 2-4294967296	120B	26.1%	44.0%	48.5%
LEANA 2-8589934592	120B	26.1%	44.0%	48.5%
LEANA 2-17179869184	120B	26.1%	44.0%	48.5%
LEANA 2-34359738368	120B	26.1%	44.0%	48.5%
LEANA 2-68719476736	120B	26.1%	44.0%	48.5%
LEANA 2-137438953472	120B	26.1%	44.0%	48.5%
LEANA 2-274877906944	120B	26.1%	44.0%	48.5%
LEANA 2-549755813888	120B	26.1%	44.0%	48.5%
LEANA 2-1099511627776	120B	26.1%	44.0%	48.5%
LEANA 2-2199023255552	120B	26.1%	44.0%	48.5%
LEANA 2-4398046511104	120B	26.1%	44.0%	48.5%
LEANA 2-8796093022208	120B	26.1%	44.0%	48.5%
LEANA 2-17592186044416	120B	26.1%	44.0%	48.5%
LEANA 2-35184372088832	120B	26.1%	44.0%	48.5%
LEANA 2-70368744177664	120B	26.1%	44.0%	48.5%
LEANA 2-140737488355328	120B	26.1%	44.0%	48.5%
LEANA 2-281474976710656	120B	26.1%	44.0%	48.5%
LEANA 2-562949953421312	120B	26.1%	44.0%	48.5%
LEANA 2-1125899906842624	120B	26.1%	44.0%	48.5%
LEANA 2-2251799813685248	120B	26.1%	44.0%	48.5%
LEANA 2-4503599627370496	120B	26.1%	44.0%	48.5%
LEANA 2-9007199254740992	120B	26.1%	44.0%	48.5%
LEANA 2-18014398509481984	120B	26.1%	44.0%	48.5%
LEANA 2-36028797018963968	120B	26.1%	44.0%	48.5%
LEANA 2-72057594037927936	120B	26.1%	44.0%	48.5%
LEANA 2-144115188075855872	120B	26.1%	44.0%	48.5%
LEANA 2-288230376151711744	120B	26.1%	44.0%	48.5%
LEANA 2-576460752303423488	120B	26.1%	44.0%	48.5%
LEANA 2-1152921504606846976	120B	26.1%	44.0%	48.5%
LEANA 2-2305843009213693952	120B	26.1%	44.0%	48.5%
LEANA 2-4611686018427387904	120B	26.1%	44.0%	48.5%
LEANA 2-9223372036854775808	120B	26.1%	44.0%	48.5%
LEANA 2-18446744073709551616	120B	26.1%	44.0%	48.5%
LEANA 2-36893488147419103232	120B	26.1%	44.0%	48.5%
LEANA 2-73786976294838206464	120B	26.1%	44.0%	48.5%
LEANA 2-147573952589676412928	120B	26.1%	44.0%	48.5%
LEANA 2-295147905179352825856	120B	26.1%	44.0%	48.5%
LEANA 2-590295810358705651712	120B	26.1%	44.0%	48.5%
LEANA 2-1180591620717411303424	120B	26.1%	44.0%	48.5%
LEANA 2-2361183241434822606848	120B	26.1%	44.0%	48.5%
LEANA 2-4722366482869645213696	120B	26.1%	44.0%	48.5%
LEANA 2-9444732965739290427392	120B	26.1%	44.0%	48.5%
LEANA 2-18889465931478580854784	120B	26.1%	44.0%	48.5%
LEANA 2-37778931862957161709568	120B	26.1%	44.0%	48.5%
LEANA 2-75557863725914323419136	120B	26.1%	44.0%	48.5%
LEANA 2-151115727451828646838272	120B	26.1%	44.0%	48.5%
LEANA 2-302231454903657293676544	120B	26.1%	44.0%	48.5%
LEANA 2-604462909807314587353088	120B	26.1%	44.0%	48.5%
LEANA 2-1208925819614629174706176	120B	26.1%	44.0%	48.5%
LEANA 2-2417851639229258349412352	120B	26.1%	44.0%	48.5%
LEANA 2-4835703278458516698824704	120B	26.1%	44.0%	48.5%
LEANA 2-9671406556917033397649408	120B	26.1%	44.0%	48.5%
LEANA 2-19342813113834066795298816	120B	26.1%	44.0%	48.5%
LEANA 2-38685626227668133590597632	120B	26.1%	44.0%	48.5%
LEANA 2-77371252455336267181195264	120B	26.1%	44.0%	48.5%
LEANA 2-154742504910672534362390528	120B	26.1%	44.0%	48.5%
LEANA 2-309485009821345068724781056	120B	26.1%	44.0%	48.5%
LEANA 2-618970019642690137449562112	120B	26.1%	44.0%	48.5%
LEANA 2-1237940039285380274899124224	120B	26.1%	44.0%	48.5%
LEANA 2-2475880078570760549798248448	120B	26.1%	44.0%	48.5%
LEANA 2-4951760157141521099596496896	120B	26.1%	44.0%	48.5%
LEANA 2-9903520314283042199192993792	120B	26.1%	44.0%	48.5%
LEANA 2-19807040628566084398385987584	120B	26.1%	44.0%	48.5%
LEANA 2-39614081257132168796771975168	120B	26.1%	44.0%	48.5%
LEANA 2-79228162514264337593543950336	120B	26.1%	44.0%	48.5%
LEANA 2-158456325028528675187087900672	120B	26.1%	44.0%	48.5%
LEANA 2-316912650057057350374175801344	120B	26.1%	44.0%	48.5%
LEANA 2-633825300114114700748351602688	120B	26.1%	44.0%	48.5%
LEANA 2-1267650600228229401496703205376	120B	26.1%	44.0%	48.5%
LEANA 2-2535301200456458802993406410752	120B	26.1%	44.0%	48.5%
LEANA 2-5070602400912917605986812821504	120B	26.1%	44.0%	48.5%
LEANA 2-10141204801825835211973625643008	120B	26.1%	44.0%	48.5%
LEANA 2-20282409603651670423947251286016	120B	26.1%	44.0%	48.5%
LEANA 2-40564819207303340847894502572032	120B	26.1%	44.0%	48.5%
LEANA 2-81129638414606681695789005144064	120B	26.1%	44.0%	48.5%
LEANA 2-162259276829213363915780010288128	120B	26.1%	44.0%	48.5%
LEANA 2-324518553658426727831560020576256	120B	26.1%	44.0%	48.5%
LEANA 2-649037107316853455663120041152512	120B	26.1%	44.0%	48.5%
LEANA 2-1298074214633706911326240082305024	120B	26.1%	44.0%	48.5%
LEANA 2-2596148429267413822652480164610048	120B	26.1%	44.0%	48.5%
LEANA 2-5192296858534827645304960329220096	120B	26.1%	44.0%	48.5%
LEANA 2-10384593717069655290609920658440192	120B	26.1%	44.0%	48.5%
LEANA 2-20769187434139310581219841316880384	120B	26.1%	44.0%	48.5%
LEANA 2-41538374868278621162439682633760768	120B	26.1%	44.0%	48.5%
LEANA 2-83076749736557242324879365267521536	120B	26.1%	44.0%	48.5%
LEANA 2-166153499473114484649758725535042672	120B	26.1%	44.0%	48.5%
LEANA 2-332306998946228969299517450670085344	120B	26.1%	44.0%	48.5%
LEANA 2-664613997892457938599034901340170688	120B	26.1%	44.0%	48.5%
LEANA 2-1329227995784915877198069802680341376	120B	26.1%	44.0%	48.5%
LEANA 2-2658455991569831754396139605360682752	120B	26.1%	44.0%	48.5%
LEANA 2-5316911983139663508792279210721365504	120B	26.1%	44.0%	48.5%
LEANA 2-10633823966279327017584558421442731008	120B	26.1%	44.0%	48.5%
LEANA 2-21267647932558654035169116842885462016	120B	26.1%	44.0%	48.5%
LEANA 2-42535295865117308070338233685770924032	120B	26.1%	44.0%	48.5%
LEANA 2-85070591730234616140676467371541848064	120B	26.1%	44.0%	48.5%
LEANA 2-170141183460469232281352934743083696128	120B	26.1%	44.0%	48.5%
LEANA 2-340282366920938464562705869486167392256	120B	26.1%	44.0%	48.5%
LEANA 2-680564733841876929125411738972334784512	120B	26.1%	44.0%	48.5%
LEANA 2-1361129467683753858250823477944669569024	120B	26.1%	44.0%	48.5%
LEANA 2-2722258935367507716501646955889339138048	120B	26.1%	44.0%	48.5%
LEANA 2-544451787073501543300329391177867826096	120B	26.1%	44.0%	48.5%
LEANA 2-1088903574147003086600658782355735652192	120B	26.1%	44.0%	48.5%
LEANA 2-2177807148294006173201317564711471304384	120B	26.1%	44.0%	48.5%
LEANA 2-4355614296588012346402635129422942608768	120B	26.1%	44.0%	48.5%
LEANA 2-8711228593176024692805270258845885217536	120B	26.1%	44.0%	48.5%
LEANA 2-1742245718635204938561054051769177043712	120B	26.1%	44.0%	48.5%
LEANA 2-3484491437270409877122108810383344087424	120B	26.1%	44.0%	48.5%
LEANA 2-6968982874540819754244217620766688174848	120B	26.1%	44.0%	48.5%
LEANA 2-1393796574908163950848843521553377349792	120B	26.1%	44.0%	48.5%
LEANA 2-2787593149816327901697687043106754699584	120B	26.1%	44.0%	48.5%
LEANA 2-5575186299632655803395374086213509399168	120B	26.1%	44.0%	48.5%
LEANA 2-11150372599265311606790748172427019798336	120B	26.1%	44.0%	48.5%
LEANA 2-22300745198530623213581489644854039596672	120B	26.1%	44.0%	48.5%
LEANA 2-44601490397061246427163778889708079133344	120B	26.1%	44.0%	48.5%
LEANA 2-89202980794122492854327557779416158266688	120B	26.1%	44.0%	48.5%
LEANA 2-178405961588244985708655115558832316533376	120B	26.1%	44.0%	48.5%</

Annotated table from the CodeJama paper

Why No 70B Code Llama Model?

Interestingly, Code Llama is only available as 7B, 13B, and 34B parameter versions; unlike Llama 2, there is no 70B version. While the paper doesn't provide any explanation, I suspect this could be due to two reasons:

1. The Code Llama models were trained on 500B additional code tokens, starting with Llama 2 weights, whereas Llama 2 models were trained on 2T tokens. Since the Code Llama model was trained on 4x fewer domain-specific tokens, maybe a CodeLlama 70B version did not perform well enough due to [LLM scaling laws](#)—there was not enough training data.
2. Code Llama models support context sizes of 100k, which is very useful when working with code. Llama 2, in contrast, only supports up to 4k tokens as input. These 70B models may be computationally infeasible (or reasonable hardware clusters) if they were to support 100k token inputs.

GPT-4 Model Details Have Leaked

Switching gears from open-source to closed-source models, there have also been a few interesting news on the GPT-4 front last month. For instance, the GPT-4 mode [details have now apparently \(finally\) been leaked](#):

> GPT-4 is a language model with approximately 1.8 trillion parameters across 120 layers, 10x larger than GPT-3. It uses a Mixture of Experts (MoE) model with 16 experts, each having about 11 billion parameters. Utilizing MoE allows for more efficient use of resources during inference, needing only about 280 billion parameters and 560 TFLOPs, compared to the 1.8 trillion parameters and 3,700 TFLOPs required for a purely dense model.

> The model is trained on approximately 13 trillion tokens from various sources, including internet data, books, and research papers. To reduce training costs, OpenAI employs tensor and pipeline parallelism, and a large batch size of 60 million. The estimated training cost for GPT-4 is around \$63 million.

Source: https://www.reddit.com/r/LocalLLaMA/comments/14wbmio/gpt4_details_leaked/

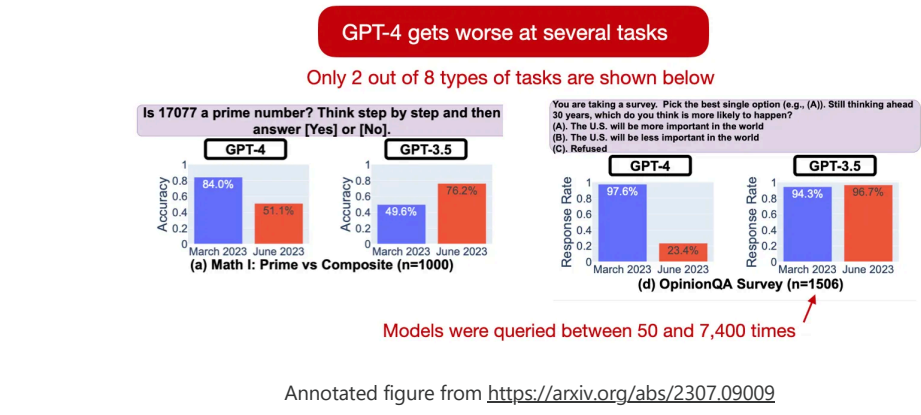
In particular, the fact that GPT-4 is apparently using a Mixture of Experts (MoE) approach is very interesting and important to highlight. MoE means that it is to improve the performance of a system by combining the predictions or decisions of several specialized sub-models or "experts."

So, suppose we want to improve models like Llama 2 further and outperform GPT-4-like offerings (by a large margin). In that case, we may not only have to scale the model to a similar size (in this case, 25x larger to be comparable) but also consider using MoE approaches.

Is GPT-4 Getting Worse?

We often see claims that OpenAI degrades the ChatGPT performance over time to save computation time and cost. In the recent [How is ChatGPT's behavior changing over time?](#) paper, researchers make an interesting observation that GPT-4's modeling performance appears to be

indeed getting worse over time. Is it due to distillation methods to save costs or guard rails to prevent certain types of misuse? (By the way, it is interesting to see research efforts being diverted into studying these changes that could likely already be answered by a researcher and engineers who worked on these models.)



Annotated figure from <https://arxiv.org/abs/2307.09002>

As a follow-up, I highly recommend reading the Substack article [@GPT-4 getting worse over time](#) by Sayash Kapoor and Arvind Narayananwho highlight important shortcomings of the research article above. For instance,

> What seems to have changed is that the March version of GPT-4 almost always guesses that the number is prime, and the June version almost always guesses that it is composite. The authors interpret this as a massive performance drop — since they only test primes. For GPT-3.5, this behavior is reversed.

So, the claim that GPT-4 is getting worse over time may not be necessarily true. There are ongoing changes with regard to its behavior, though.

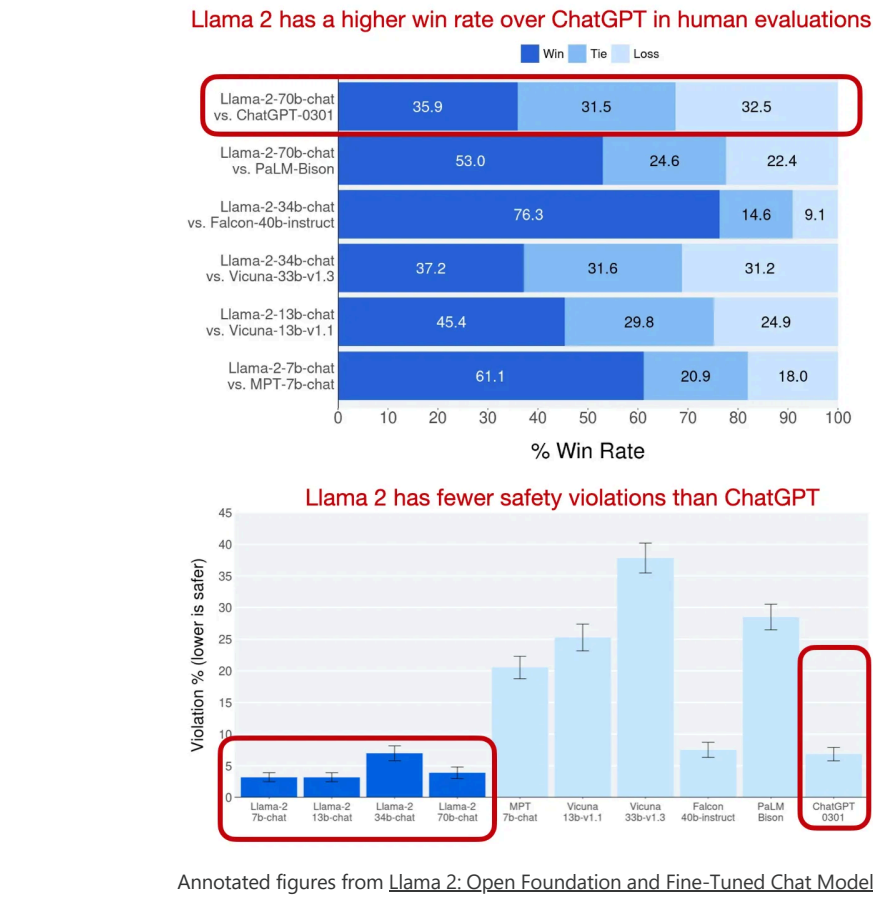
In practice, one of the biggest advantages of using an API is that we don't have to worry about hosting and serving the model itself. However, this is also the biggest shortcoming if we build services on top of it. Sure, performance improvements are usually appreciated. However, intransparent changes to the model behavior can also cause all your previously working queries to not work reliably the next morning. The more you build on top of a closed API, the more annoying this problem can become.

ChatGPT Finetuning As A Service

Last Wednesday, [OpenAI announced its new finetuning service for ChatGPT](#). This is super interesting for those who want to finetune LLMs on custom data.

It will be interesting to see how it fares against open-source solutions from which the most recent cutting-edge finetuning techniques emerged (Llama-Adapters, LoRA, QLoRA, and more).

Also, the recent open-source Llama 2 model, which we discussed above, compares very favorably to ChatGPT / GPT-3.5, as shown below.



Annotated figures from [Llama 2: Open Foundation and Fine-Tuned Chat Models](#)

Based on the OpenAI documentation and marketing materials, it doesn't seem like the service will be restricted to just changing the style or tone of the LLM's responses. Instead, it appears that OpenAI is offering a fully-fledged instruction tuning service.

However, this also means it will likely still be impossible to fully adapt the models to new specialized target domains like medical, financial, or legal contexts. That's because new knowledge is usually ingested via pretraining, not finetuning; this is also true for open-source models.

Our [GPT best practices guide](#) provides a background on some of the most effective strategies and tactics for getting better performance without fine-tuning. You may find it helpful to iterate quickly on prompts in our playground.

Common use cases

Some common use cases where fine-tuning can improve results:

- Setting the style, tone, format, or other qualitative aspects
- Improving reliability at producing a desired output
- Correcting failures to follow complex prompts
- Handling many edge cases in specific ways
- [Performing a new skill or task that's hard to articulate in a prompt](#)

Excerpt from the [OpenAI documentation](#)

While OpenAI does not specify which finetuning method they will use, opting for low-rank adaptation (LoRA) as a finetuning technique would make sense. I believe this would be the easiest and most cost-effective because they won't have to store a new 175B parameter LLM for each customer. With LoRA you can use the same base LLM but only need to store the LoRA weights.

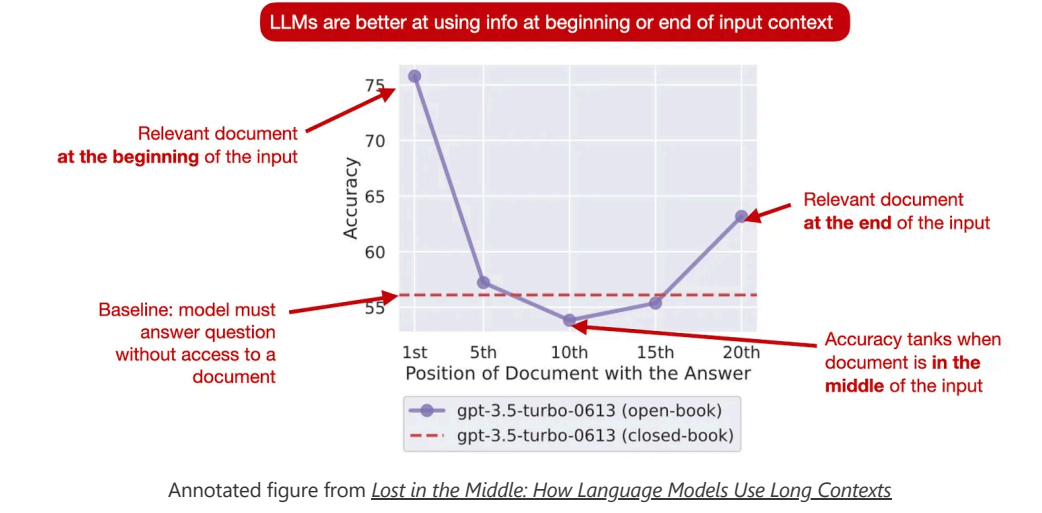
Scaling LLMs and Alternatives to Transformer-based LLMs

One of the open research challenges is how to extend LLMs to larger input contexts. Recent approaches include

- 1) the [RMT paper on scaling transformers to 1 million tokens](#);
- 2) the [convolutional Hyena LLM for 1 million tokens](#);
- 3) [LongNet scaling transformers to 1 billion tokens](#).

While there are several use cases for such long LLMs, for example, asking questions about particular long document inputs, the elephant in the room is: How well do LLMs use these longer contexts?

New research ([Lost in the Middle: How Language Models Use Long Contexts](#)) shows that LLMs are good at retrieving information at the beginning of documents. They do less well in terms of retrieving information if it's contained in the middle of a document.



Annotated figure from [Lost in the Middle: How Language Models Use Long Contexts](#)

The analysis in this paper is focused on ChatGPT (as it's shown in the figure) and Claude. Of course, it would be interesting to include other models (Hyena, LongNet etc.) in the future.

Nonetheless, this is quite interesting!

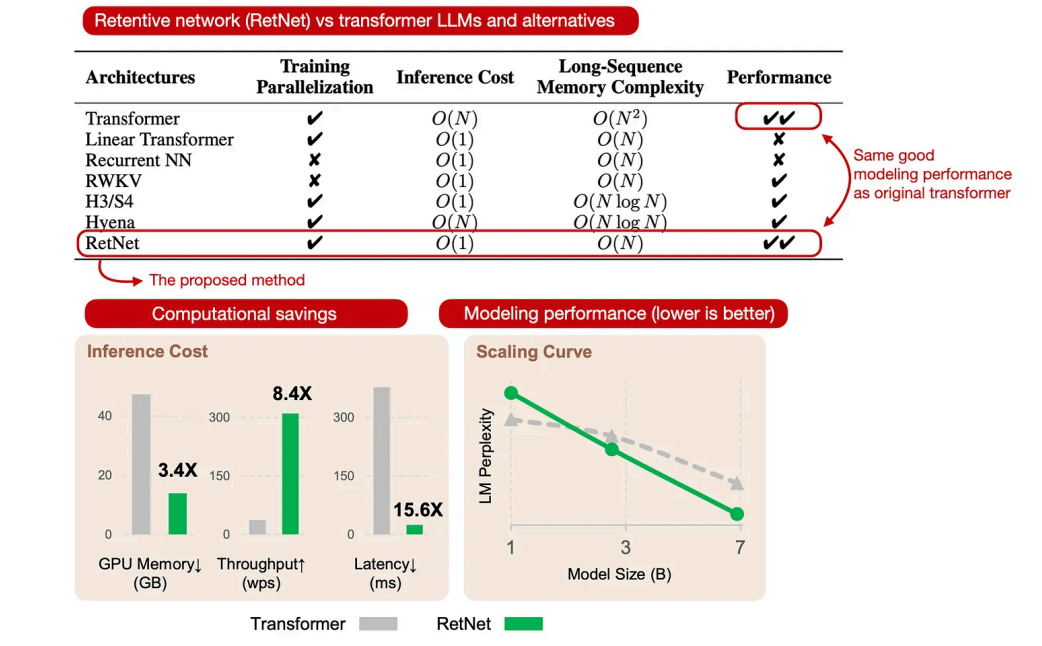
- 1) I would expect that the opposite is true for, e.g., RNN-based LLMs like [RWKV](#) (since it's processing information sequentially, it might rather forget early information)
- 2) To my knowledge, there is no specific inductive bias in transformer-based LLM architectures that explains why the retrieval performance should be worse for text in the middle of the document. I suspect it is all because of the training data and how humans write: the most important information is usually in the beginning or the end (think paper Abstracts and Conclusion sections), and it's then how LLMs parameterize the attention weights during training.

More Transformer Alternatives

I mentioned several alternatives to transformer-based LLMs in the paragraph above, such as the recurrent [RWKV](#) LLM and the convolutional [Hyena LLM](#).

The newest transformer alternative is Retentive Network (also called RetNetNet), which was proposed in a paper last month with a bold headline: [Retentive Network: A Successor to Transformer for Large Language Models](#).

This paper proposes another alternative to large language transformers, one that has linear instead of quadratic complexity concerning the input sequence lengths. RetentiveNet can be trained in a parallel mode and switched to a recurrent model to extend context lengths without increasing memory cost to achieve good inference performance. The largest model was 6.7B parameters, and it will be interesting to see future studies comparing RetNet to Llama-2 70B and others.



Will it be a widely used alternative to transformer-based LLMs? After the initial hype, it doesn't seem like it's being adopted in any other projects at the moment, but time will tell.

What are "Frontier AI Models"?

The AI community loves coining new terms!

For quite a while, I found myself resistant to using the term "foundation model." I was convinced that there was no need to introduce a new phrase to describe pretrained large language models (LLMs) or pretrained vision transformers (ViTs).

Initially, many people saw the paper that coined the term "foundation model" appeared to provide academics as a means to garner citations. (This sentiment was evidenced by the [113 author Stanford article that coined this term](#), collecting 1372 citations as of today.) However, despite early criticisms, the term has started to gain more widespread acceptance in recent times. In keeping with this trend, I've also begun incorporating "foundation models" into my vocabulary.

Last month, authors from the [Centre for the Governance of AI](#) (with co-authors from Google DeepMind, OpenAI, and Microsoft) wrote a new paper, [Frontier AI Regulation: Managing Emerging Risks to Public Safety](#), to coin a new term "Frontier AI models" on top of foundation models:

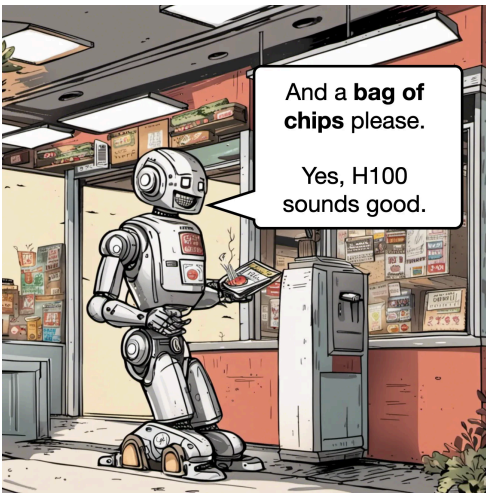
> We define "frontier AI models" as highly capable foundation models that could exhibit dangerous capabilities.

We live in interesting times.

Hardware Corner

Despite or because of all the recent LLM progress, there's been an ongoing GPU shortage, especially when it comes to NVIDIA's H100 chips. As VentureBeat recently reported, [Nvidia GPU shortage is 'non-possible' of Silicon Valley](#).

While everyone is fighting over H100, NVIDIA also announced its next generation of GPUs, [the GH200](#), which is going to offer 141 GB of RAM (as opposed to 80 GB in the H100) -- maybe another way of solving the LLM long-context problem with brute (hardware) force.



Machine learning humor

LLMs and Copyright Laws

What is the current status on LLMs and generative AI regarding copyright laws? 8 months into 2023, it seems that this question still doesn't have (many) definitive answers. Below, I polled a few articles based on the best of my knowledge. However if you have any additional insights or resources, I'd love to hear! Note that I am not a lawyer and cannot comment on any of the legal aspects. Also, since I am not a lawyer, the following information should not be taken as legal advice.

According to Wikipedia, "The copyright status of LLMs trained on copyrighted material is not yet fully understood."

In this context, a recent article and study that appeared in the Atlantic, [Revealed: The Authors Whose Pirated Books Are Powering Generative AI](#) is worth highlighting.

This article addresses the recent legal action taken by various authors against Meta, accusing the company of utilizing their copyrighted works in the training of [Llama 3](#). The reason for this lawsuit was the training data, which included Book's section of [The Pile](#), encompassing no less than 170,000 books. Notably, the same dataset was used for EleutherAI's [GPT-J](#) (a model similar to GPT-3) and [BloombergGPT](#), Bloomberg's language model for finance, which was detailed in the third issue of Ahead of AI.

Moreover, [NPR recently reported](#) that the New York Times is preparing a lawsuit against OpenAI since it's concerned that ChatGPT generates text that responds to inquiries, drawing from the initial reporting and writing produced by the paper's team.

Laws are laws, and developers should work with content creators to avoid violating copyright laws. In addition, it would be helpful if there were updated legal guidelines that clearly address the use of LLMs. For drafting policies or amending laws, it could be helpful to consider "fair use" contexts. In other words, it may also be helpful to distinguish between LLMs developed for commercial purposes (for example, ChatGPT) and LLMs that are solely developed for research purposes whose licenses forbid commercial applications (for example, Meta's LLaMA).

What is "fair use"? According to Wikipedia, in the United States, the principle of "fair use" within copyright law may allow the utilization of copyrighted materials without authorization, provided that the use involves a substantial transformation of the work and does not endanger the copyright owner's interests. However, whether this concept of fair use extends to the training of machine learning models is still unresolved.

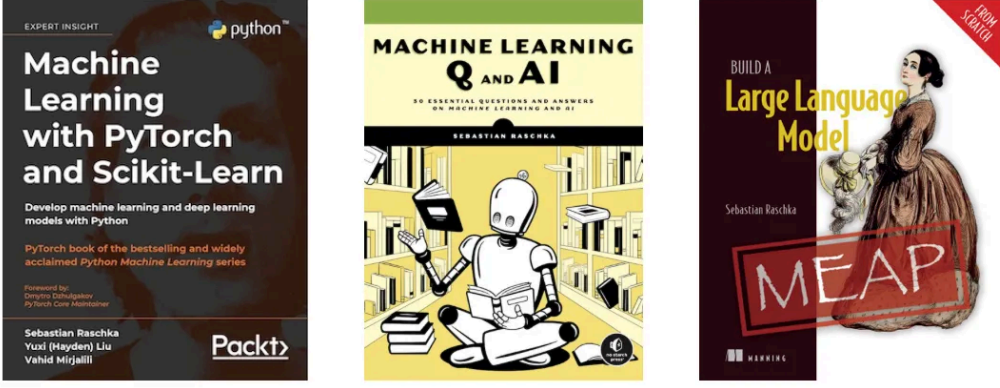
How do other countries handle the use and training of AI concerning copyrighted materials?

A few months ago, [PetaPixel and other news websites reported](#) that Japan declared that it would permit generative AI training on any kind of data and that it will not enforce copyrights when it comes to training generative AI models.

The AI Act in the EU requires companies to publicly disclose any copyrighted work used in training, as reported by [Lexology](#). However, this article is not very clear on what happens if copyrights are violated.

Overall, it seems that a lot of rules are still being drafted and amended. I am hoping that the rules, whatever they are, remain clear so that AI researchers and practitioners can adjust and act accordingly.

This magazine is personal passion project that does not offer direct compensation. However, for those who wish to support me, please consider purchasing a copy of one of my books. If you find them insightful and beneficial, please feel free to recommend them to your friends and colleagues.



Machine Learning with PyTorch and Scikit-Learn, Machine Learning Q and A, and Build a Large Language Model from Scratch

Your support means a great deal! Thank you!

111 Likes 6 Retacks

14 Comments

- Sevika Kudin · Aug 27, 2023 · [Liked by Sebastian Raschka, PhD](#)
It's a real pleasure to read the summary of such a knowledgeable man as you, kudos!
Now, I'm a little bit surprised that you didn't introduce multi-modality in LLMs as a main axis of research/differentiation. Pairing text to vision is already relatively straightforward and there is also sub-modality differentiation in the sound landscape with speech to text enlarged to text to sound (e.g. the Bark model), but this is only the beginning.
IMHO, generalized multi-modality it is a too neglected straightforward path towards AGI as it would solve a good part o the thorny issue of symbol grounding in those models (the other parts being feedback from retro-action from the world to the models, a direct pathway toward the synthesis of genuine evolved intentionality).
With models encompassing our 5 senses and proprioception, they should evolve inner world representations more aligned with human ones. One can even speculate if those LLMs would converge toward an universal underlying neural coding scheme like e.g. a kind of Grossberg ART refinement, as described in this paper: <https://www.mpsg.com/2078-2899/14/2/36>

👍

👎

🗨

🔄

🗑

1 reply by Sebastian Raschka, PhD

Milton Leal · Aug 26, 2023 · [Liked by Sebastian Raschka, PhD](#)
Awesome write-up. I tend to try to follow the news as they occur, but you do such a great job in distilling everything that I may just consume your newsletter. I wonder if you use any LLM to help you writing or organizing new text.

👍

👎

🗨

🔄

🗑

1 reply by Sebastian Raschka, PhD

12 more comments...

© 2024 Sebastian Raschka · Privacy · Terms · Collection notice
Substack is the home for great culture

https://magazine.sebastianraschka.com/p/ahead-of-ai-11-new-foundation-models

4/4